Guiding Questions — Answers (Layman + Technical)

Preparation for the poster presentation and discussion (Deep Learning for Maritime Vision Applications, SS 2026). **Part A** = lecture concepts (explained in our own words). **Part B** = our AutoFish fish-length-estimation project, argued from our ODE, poster, and numbers.

Each answer gives a **plain-language** version and a **technical** version.

Team: Abu Bakar, Laksh Jiwani, Shahman Butt · Repo: https://github.com/Shahman-Butt/AreaSeminarFishLength

---

PART A — Lecture material

A1. Why does a neural network need a non-linear activation between layers? What collapses without it?

Plain: Without a non-linearity, stacking layers is pointless — many layers behave like a single one, so the network can only draw straight-line boundaries and can't learn curves or complex shapes. **Technical:** A linear layer is $Wx+b$. Composing linear maps is still linear: $W_2(W_1x)=(W_2W_1)x$. So without a non-linearity (ReLU, GELU, …) the whole network **collapses to one linear transformation**, no matter how deep. Non-linearities give the network the capacity to approximate arbitrary functions.

A2. Why learn CNN kernels from data instead of hand-crafting (e.g. Sobel)? Early vs deep layers?

Plain: A hand-made filter (like Sobel for edges) only does one fixed thing. Letting the network learn its filters means it discovers whatever patterns actually help the task, and it can learn thousands of them. Early layers learn simple things (edges, colors); deep layers combine those into complex things (textures, fish body shapes). **Technical:** Learned kernels are optimized end-to-end for the loss, so features are task-adapted rather than generic. Empirically, early conv layers respond to oriented edges / color blobs (like Gabor/Sobel filters, but learned); deeper layers compose these into textures, parts, and object-level concepts. This hierarchy is what transfers well when we fine-tune ImageNet encoders on fish.

A3. What makes a CNN better suited to images than a fully connected network, and what does each buy you?

Plain: CNNs slide the same small filter across the whole image, so they need far fewer parameters, and they recognize a pattern wherever it appears. **Technical:** (1) **Local connectivity** — each neuron sees a small receptive field → far fewer weights/compute than dense layers. (2) **Weight sharing** — the same kernel is reused across all positions → parameter efficiency + **translation equivariance**. (3) **Pooling / hierarchy** → some **translation invariance** and multi-scale features. Net effect: fewer parameters, less computation, and built-in spatial priors that a fully connected net would have to learn from scratch.

A4. CNN vs Transformer — how does each move information across the image, and what does that mean for data needs?

Plain: A CNN builds up understanding locally, patch by patch, so a far-apart relationship only forms after many layers. A Transformer lets every patch look at every other patch immediately, so it sees global relationships early — but that freedom means it needs much more data to learn good habits. **Technical:** CNNs propagate information through **local receptive fields**; long-range dependencies require depth/pooling. Transformers use **self-attention** — every token attends to every token in one layer (global receptive field), at $O(N^2)$ cost. CNNs

have a strong locality/translation **inductive bias** (data-efficient); ViTs have weaker priors, so they need **large-scale pretraining** (or foundation-model weights) to compete — which is exactly why we used pretrained DINOv2/CLIP rather than training a ViT from scratch on 11k crops.

A5. What are the core computer-vision tasks, how do outputs differ, and what does that mean for annotation cost?

Plain: Classification says *what* (one label), detection says *where* (a box), segmentation says *which pixels* (an outline), regression says *how much* (a number). The more precise the output, the more expensive the labels. **Technical:** Classification → class label (cheap). Detection → bounding boxes (moderate). Segmentation → per-pixel masks (expensive — pixel annotation). Regression → continuous value (label cost depends on measurement, here a physical length). **Our task is regression** (length in cm); the dataset also provides masks (used only for cropping) and species labels (used for the classification sub-task).

A6. What is the [CLS] token, why use it for classification, and why use patch tokens for segmentation/depth?

Plain: A ViT turns an image into a grid of patch "tokens" plus one extra summary token (CLS). CLS is a whole-image summary — great for "what is this?". But for tasks that need *where* things are (segmentation, depth, or precise size), you need the per-location patch tokens, because CLS has thrown the spatial detail away. **Technical:** [CLS] is a learned token that aggregates global information via attention → a single image-level embedding, ideal for classification. Patch tokens retain **spatial structure** (one vector per image region), needed for dense prediction. **This is central to our project:** for DINOv2 we first used the CLS token (poor for length), then switched to **mean-pooled patch tokens** — which reliably improved length regression (1.843 → 1.261 cm over 3 seeds), because length is a geometry/size task that benefits from spatial detail.

A7. Three phases of a foundation model; pretext task of MAE vs DINO; why does MAE mask so many patches?

Plain: (1) Pretrain on huge unlabeled data with a made-up "pretext" task; (2) benchmark the frozen features; (3) apply to a real downstream task. MAE hides most of the image and asks the model to paint it back; DINO makes two views of an image agree. MAE hides a lot so the task is hard enough to force real understanding rather than trivial copying. **Technical:** Phases: **pretraining** (self-supervised) → **benchmarking** (linear probe / kNN on frozen features) → **application** (fine-tune or probe on a target task). **MAE** pretext = masked autoencoding: reconstruct missing patches (typically ~75% masked). High masking ratio removes redundancy so the model can't just interpolate neighbors — it must learn semantic structure. **DINO** pretext = self-distillation: a student network matches a teacher's output on different augmented views (no labels), yielding features with strong emergent object structure.

A8. What does linear probing measure, and why must the backbone stay frozen? What can full fine-tuning hide?

Plain: Linear probing = freeze the big model, train only a tiny classifier on top. It measures how good the *pretrained features already are*. If you instead fine-tune everything, a great final score might be thanks to the fine-tuning, not the pretrained features — hiding how good (or bad) the features really were. **Technical:** Linear probing trains a single linear layer on frozen features → an honest measure of **representation quality/linear separability**. The backbone must stay frozen or you're no longer measuring the pretrained representation. Full fine-tuning can **mask weak pretraining**: even mediocre features can reach high accuracy once the backbone adapts, so it conflates representation quality with adaptation capacity. In our study, comparing **frozen vs fine-tuned** made this explicit — frozen DINOv2 was weak (1.74 cm), and adaptation only partly closed the gap.

A9. Three ways to adapt a frozen VFM, ordered by adaptation, and when to choose each.

Plain: (1) Train only a small head on frozen features — cheapest, safest on little data. (2) Unfreeze the last block or two — a middle ground. (3) Fine-tune the whole model — most powerful but needs lots of data or it overfits/forgets. **Technical:** Increasing adaptation: **linear/MLP probe (frozen)** < **partial fine-tuning (last block(s), or PEFT like LoRA/adapters)** < **full fine-tuning**. Choose frozen for small datasets or when you want an honest feature benchmark; partial when you have moderate data and want task adaptation without destabilizing pretraining; full when data is plentiful. **We tried all three on DINOv2:** frozen (1.74), last-block (1.44), full FT (1.78–2.13) — partial was best among CLS variants, and full FT was unstable on ~11k crops, exactly as this ordering predicts.

A10. What must be disclosed about generative-AI use, and what responsibility stays with you?

Plain: You must say **where** and **how** you used AI tools, and you remain fully responsible for correctness — the AI is a helper, not an author. **Technical:** Per the course policy, disclose the tools, where they were applied (code scaffolding, text drafting, figure captions) and how (language improvement, summarization, generation). The student retains **full oversight and factual-accuracy responsibility**; undisclosed/unverified AI content fails the course. **Our disclosure:** generative AI assisted with code scaffolding and drafting of documentation/figures; all experiments, numbers, and conclusions were produced by our pipeline and verified by us against saved metric files.

PART B — Our project

B1. Which models did you use, and each one's role?

Plain: MobileNetV2 is the paper's baseline we reproduce. EfficientNet-B0 and ConvNeXt-Tiny are newer supervised CNNs (challengers). CLIP and DINOv2 are the vision foundation models we test. Each replaces only the "eye" of the same system so we can compare fairly. **Technical:** Encoders (all with the same bbox+MLP regression head): **MobileNetV2** (reproduced baseline), **EfficientNet-B0**, **ConvNeXt-Tiny** (supervised-ImageNet CNNs), **CLIP ViT-B/32** and **DINOv2 ViT-S/14** (vision foundation models). Roles: baseline vs supervised-CNN challengers vs foundation-model challengers, in a controlled encoder-swap comparison.

B2. How do the two model families process an image differently? How many params trained vs frozen?

Plain: CNNs scan with small sliding filters; transformers cut the image into patches and let them all talk to each other. In frozen experiments we trained only the small head; in fine-tuning we trained the whole encoder. **Technical:** CNNs = local convolutions + hierarchy; ViTs = patch embedding + global self-attention. Approx. parameter counts: MobileNetV2 ≈ 3.4M, EfficientNet-B0 ≈ 5.3M, ConvNeXt-Tiny ≈ 28M, DINOv2 ViT-S/14 ≈ 21M, CLIP ViT-B/32 image encoder ≈ 88M. **Trained vs frozen:** full fine-tune → all encoder params + head trained; **frozen** → encoder frozen, only the MLP head (~0.7M) trained; **last-block** → only the final transformer block + norm/proj + head trained (a few M).

B3. What was each model pretrained on (data, objective, labels)? Why does it matter for our images?

Plain: The CNNs learned from ImageNet with labels; CLIP learned from web image–caption pairs; DINOv2 taught itself from images with no labels. What they learned shapes how well they transfer to top-down fish photos. **Technical:** MobileNetV2/EfficientNet-B0/ConvNeXt-Tiny — **ImageNet-1k, supervised** (labels). CLIP ViT-B/32 — ~400M **image–text pairs, contrastive** (weak language supervision). DINOv2 ViT-S/14 — **self-supervised** (no

labels, DINO/iBOT). Matters because our images are out-of-distribution top-view masked fish; supervised-ImageNet features (edges→shapes→size cues) transfer better to precise **metric** regression than semantics-oriented self-supervised CLS features — consistent with our ranking.

B4. What is one sample? What is the prediction target and its unit?

Plain: One sample is one fish in one photo (a masked, cropped picture of that fish). The target is that fish's length, in centimetres. **Technical:** One sample = one annotation = a 224×224 masked square crop of a single fish instance + its 4 normalized bbox values. Target = `length_cm` (continuous, centimetres). 18,157 samples total; test = 3,759.

B5. How did you split the data, and what could go wrong with a naive per-image random split?

Plain: We split by fish groups (15 train / 5 val / 5 test), never by photo. A random per-photo split would put the same fish in both training and test, so the model could just recognize that individual fish and look better than it really is. **Technical:** Official **group-level split** — all appearances of a fish stay on one side. A per-image random split causes **identity leakage** (same `fish_id` in train and test) → optimistic, invalid test error. We audited and removed one cross-split fish (id 113) so leakage = 0; test non-occluded count is 1,879 (not 1,880) as a result.

B6. What is your primary metric and why? Which metric would look good but hide the failure you care about?

Plain: Our main metric is the average centimetre error (MAE) — it's directly meaningful and matches the paper. A metric like R^2 can look great (0.95) while the model is still off by ~0.8 cm; and a single overall number can hide that occluded fish are much harder. **Technical:** Primary = **MAE (cm)** — directly interpretable, comparable to the paper, robust for regression. Misleading alternatives: **R^2** (0.947 sounds excellent but corresponds to ~0.77 cm error); **overall MAE** hides the occluded-subset degradation (0.633 non-occluded → 0.909 occluded). We therefore always report full/non-occluded/occluded, plus RMSE/MAPE/bias.

B7. Apart from the one thing you compare, what else differs between approaches?

Plain: We only meant to change the encoder, but the learning rate also differs by encoder type (foundation models need gentler tuning), and the baseline used more epochs. We're honest about that. **Technical:** Held identical: data, split, crops, bbox input, head design, L1 loss, metrics. **Differs by necessity:** learning rate (baseline 1e-3; swaps 1e-4; DINOv2 full-FT 1e-5/1e-6), epochs/batch (baseline 200/32; swaps 100/16). This is why we (a) clarified it on the poster and (b) launched a recipe-matched search to remove the confound for the strongest challengers.

B8. Pick one sample your model gets wrong. What makes it hard?

Plain: DINOv2 badly over-predicts small, wide flatfish — e.g. a 22.5 cm flatfish predicted as ~38 cm — probably because it confuses the fish's width/area for length. MobileNetV2 struggles most at the extremes (very small or very large fish). **Technical:** From the saved predictions, DINOv2's largest errors are small `other`-species flatfish (true 22–25 cm, predicted 34–38 cm; see `results/qualitative/mobilenet_vs_dino.png`) — a hypothesis is that its global CLS feature encodes area/shape rather than metric length. Error-by-length shows a U-shape: the shortest and longest quintiles are hardest for every model (extreme values, fewer samples).

B9. Which preprocessing step mattered most, and why?

Plain: Two things: cutting a **square, masked** crop of just the fish (so the shape isn't distorted and other fish don't interfere), and feeding the **bounding-box size** so the model knows the real scale after resizing. **Technical:** **Segmentation-masked square crops** (aspect-ratio preserving, background removed) most directly affect a

length task — non-square resizing would distort length, and masking isolates the target fish under occlusion. The **normalized bbox input** restores absolute scale lost in the 224×224 resize; it is a strong shared prior used by all encoders (and by the original paper).

B10. If you had four more weeks, what would you try and what would you hope to learn?

Plain: Finish tuning EfficientNet-B0 with a proper recipe (it's only 0.010 cm from the baseline, so a better recipe should beat it with a single model), repeat the best models with several seeds to be sure, try patch-features for CLIP, test bigger models, and add a segmentation task. **Technical:** (1) Complete the **validation-selected recipe search** (cosine LR + weight decay + tuned LR + 200 ep) for EfficientNet-B0/ConvNeXt — hypothesis: crosses the 0.771 baseline as a single model. (2) **Multi-seed** the top models for significance. (3) **Patch pooling for CLIP**; **EfficientNet-B2** and larger ViTs. (4) **Mask segmentation** (lightweight IoU-based, same encoders) to test whether the CNN-vs-VFM trend holds on dense prediction. Goal: turn "closest" into a reliable single-model win and test generality across tasks.

AI-use disclosure (per course policy)

Generative AI tools were used for **code scaffolding** (boilerplate training/evaluation/plotting code), **text drafting and language improvement** (documentation, figure captions, this Q&A), and **layout of figures/poster**. They were **not** used to generate experimental results. All models were trained by our pipeline; every number in this document is read from saved metric files (`runs/*/test_metrics.json`) and was verified by the authors, who retain full responsibility for factual accuracy.